

# 5

---

## Pola Algoritma Iteratif

Pada kuliah sebelumnya kita telah mempelajari struktur pengulangan `while`, `do-while`, dan `for`. Kita juga telah melihat bahwa pengulangan memungkinkan suatu blok instruksi dijalankan berkali-kali selama kondisi tertentu masih terpenuhi.

Pada kuliah ini, dengan pemahaman kontrol alur program melalui percabangan dan pengulangan, fokus kita adalah kombinasi pengetahuan tersebut menjadi **pola algoritma iteratif**. Tujuannya untuk mengenali bentuk-bentuk pemecahan masalah yang sering muncul berulang kali dalam komputasi sains dan rekayasa.

Banyak program yang tampak berbeda sebenarnya mempunyai struktur algoritmik yang serupa. Sebagai contoh, menghitung jumlah total energi, menghitung banyak data sensor yang melewati ambang, mencari temperatur maksimum, dan mengulangi perhitungan sampai galat cukup kecil semuanya dapat dipahami sebagai pola iteratif.

Pola-pola yang akan dipelajari cukup beragam. Daftar berikut merangkum pola utama yang akan digunakan sepanjang kuliah ini.

- ▶ pencacah (*counter*);
- ▶ pengumpul (*akumulator/accumulator*);
- ▶ nilai minimum dan maksimum;
- ▶ *sentinel-controlled iteration*;
- ▶ validasi masukan;
- ▶ pengulangan bersarang;
- ▶ pembuatan tabel komputasi;
- ▶ pembaruan keadaan secara iteratif;
- ▶ kriteria penghentian;
- ▶ pendekatan numerik sederhana.

Pengenalan pola membantu kita merancang algoritma dengan lebih sistematis. Daripada menulis pengulangan secara coba-coba, kita dapat memulai dengan memikirkan pola iteratif apa yang paling sesuai dengan masalah yang sedang dihadapi.

## 5.1 Keadaan dalam algoritma iteratif

Setiap algoritma iteratif memiliki **keadaan** (*state*) yang berubah dari satu iterasi ke iterasi berikutnya. Keadaan tersebut direpresentasikan oleh sekumpulan variabel yang menyimpan informasi penting mengenai proses yang sudah berlangsung.

Sebagai contoh, dalam algoritma penjumlahan data kita dapat memiliki variabel **sum** dan **i**. Variabel **sum** menyimpan jumlah sementara, sedangkan **i** menyimpan banyak data yang sudah diproses atau posisi iterasi saat ini.

Secara matematis, setelah data ke- $i$  selesai diproses, **sum** harus mempunyai makna yang jelas. Hubungan yang kita harapkan dapat dinyatakan sebagai berikut.

$$\text{sum} = \sum_{k=1}^i x_k.$$

Hubungan semacam ini membantu kita memahami apakah pembaruan keadaan dilakukan dengan benar. Gagasan tersebut berkaitan dengan konsep **invarian** yang telah diperkenalkan pada Kuliah 2.

### 5.1.1 Tiga komponen utama iterasi

Sebagian besar algoritma iteratif memiliki tiga komponen dasar. Ketiganya adalah keadaan awal, aturan pembaruan, dan kondisi penghentian.

Secara abstrak, struktur iterasi dapat diringkas menjadi pola yang sederhana. Bentuk umumnya dapat ditulis sebagai berikut.

```
inisialisasi keadaan
WHILE kondisi masih terpenuhi
    lakukan proses
    perbarui keadaan
END WHILE
```

Jika salah satu dari tiga bagian tersebut salah, algoritma dapat memberikan hasil keliru atau bahkan tidak pernah berhenti. Oleh karena itu, perancangan iterasi tidak cukup hanya dengan mengetahui sintaks **while** atau **for**.

## 5.2 Pola *counter*

**Counter** adalah variabel yang digunakan untuk menghitung banyak kejadian tertentu. Nilainya biasanya bertambah satu setiap kali suatu kondisi terpenuhi.

Misalkan kita ingin menghitung banyak data temperatur yang melampaui suatu batas. Kondisi yang ingin kita hitung dapat dinyatakan secara matematis sebagai berikut.

$$T > 30^{\circ}\text{C}.$$

Pola counter tersebut dapat dinyatakan dengan pseudocode sederhana. Bentuknya dapat ditulis sebagai berikut.

```
count_high ← 0
FOR setiap temperatur T
    IF T > 30
        count_high ← count_high + 1
    END IF
END FOR
```

Nilai awal harus nol karena sebelum data diproses belum ada kejadian yang dihitung. Setiap kali kondisi benar, counter diperbarui satu kali.

### 5.2.1 Implementasi dalam C

```
#include <stdio.h>

int main(void)
{
    int n;

    printf("Number of measurements: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Number of measurements must be positive.\n");
        return 1;
    }

    int count_high = 0;

    for (int i = 1; i <= n; i++)
    {
        double temperature;

        printf("T%d (C): ", i);
        scanf("%lf", &temperature);

        if (temperature > 30.0)
        {
            count_high++;
        }
    }

    printf("Measurements above 30 C = %d\n",
        count_high);

    return 0;
}
```

Program tersebut tidak perlu menyimpan seluruh data temperatur. Program hanya mempertahankan informasi yang diperlukan, yaitu jumlah data yang memenuhi kondisi.

### 5.2.2 Menghitung persentase

Counter sering digunakan bersama jumlah total data. Dari dua informasi tersebut kita dapat menghitung persentase kejadian.

Jika terdapat  $N$  data dan  $N_h$  data memenuhi kondisi, kita dapat membentuk persentase kejadian. Hubungan matematisnya dapat dituliskan sebagai berikut.

$$p = \frac{N_h}{N} \times 100\%.$$

Hubungan tersebut dapat diterjemahkan langsung ke dalam C. Salah satu bentuk implementasinya ditunjukkan berikut ini.

```
double percentage =
    100.0 * count_high / n;
```

Angka `100.0` ditulis sebagai bilangan *floating-point* agar hasil pembagian tidak mengalami pembagian integer. Ini merupakan contoh bahwa pola algoritma tetap harus dipadukan dengan pemahaman tipe data.

## 5.3 Pola akumulator

**Akumulator** adalah variabel yang digunakan untuk mengumpulkan nilai secara bertahap. Pola ini sangat sering muncul ketika kita menghitung jumlah, energi total, massa total, integral diskrit sederhana, atau besaran kumulatif lainnya.

Untuk menghitung

$$S = \sum_{i=1}^N x_i,$$

kita menggunakan

```
sum ← 0
FOR i ← 1 TO N
    sum ← sum + xi
END FOR
```

Nilai awal `sum` harus nol karena nol merupakan elemen identitas untuk penjumlahan. Jika nilai awalnya salah, seluruh hasil akhir juga akan bergeser.

### 5.3.1 Implementasi dalam C

```
#include <stdio.h>

int main(void)
{
    int n;
```

```

printf("Number of data: ");
scanf("%d", &n);

if (n <= 0)
{
    printf("N must be positive.\n");
    return 1;
}

double sum = 0.0;

for (int i = 1; i <= n; i++)
{
    double x;

    printf("x%d: ", i);
    scanf("%lf", &x);

    sum += x;
}

printf("Sum = %.6f\n", sum);

return 0;
}

```

Setelah iterasi ke- $i$ , nilai `sum` mewakili jumlah semua data yang telah diproses sampai saat itu. Hubungan ini merupakan invarian yang sangat berguna untuk memeriksa kebenaran algoritma.

## 5.4 Rata-rata sebagai gabungan akumulator dan *counter*

Rata-rata merupakan contoh sederhana yang menggabungkan akumulator dan counter. Kita memerlukan jumlah seluruh nilai serta banyaknya data yang diproses.

Secara matematis,

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i.$$

Jika  $N$  sudah diketahui sebelumnya, kita dapat menggunakan `for` tanpa counter tambahan. Jika jumlah data belum diketahui dan proses berhenti menggunakan *sentinel*, counter perlu diperbarui selama proses berlangsung.

### 5.4.1 Kasus jumlah data diketahui

```

double sum = 0.0;

for (int i = 1; i <= n; i++)
{
    double x;

```

```
scanf("%lf", &x);  
  
sum += x;  
}  
  
double mean = sum / n;
```

Dalam struktur tersebut, `n` sudah tersedia dari awal. Oleh karena itu, kita tidak memerlukan counter tambahan selain variabel pengulangan.

### 5.4.2 Kasus jumlah data tidak diketahui

Jika jumlah data tidak diketahui, kita dapat menggunakan `sum` dan `count` untuk menyimpan jumlah kumulatif serta banyak data valid. Setiap kali satu data valid dibaca, kedua variabel tersebut diperbarui.

```
double sum = 0.0;  
int count = 0;
```

Setiap data valid harus memperbarui dua keadaan sekaligus. Pembaruannya dapat ditulis sebagai berikut.

```
sum += x;  
count++;
```

Setelah proses selesai, kita harus memastikan terdapat setidaknya satu data valid. Syarat yang harus dipenuhi sebelum rata-rata dihitung adalah sebagai berikut.

$$\text{count} > 0.$$

Pemeriksaan tersebut penting untuk mencegah pembagian dengan nol. Kasus tidak adanya data valid merupakan contoh kasus batas yang harus dirancang sejak awal.

## 5.5 Pola minimum dan maksimum

Pola minimum dan maksimum digunakan untuk mencari nilai ekstrem dari sekumpulan data. Pola ini tampak sederhana, tetapi kesalahan pada inisialisasi dapat menghasilkan hasil yang salah.

Misalkan kita ingin mencari nilai minimum dan maksimum dari seluruh data. Secara matematis, kedua besaran tersebut dapat dinyatakan melalui hubungan berikut.

$$x_{\min} = \min(x_1, x_2, \dots, x_N)$$

dan

$$x_{\max} = \max(x_1, x_2, \dots, x_N).$$

Strategi yang aman adalah menggunakan data pertama sebagai nilai awal. Setelah itu, data berikutnya dibandingkan dengan nilai minimum dan maksimum sementara.

### 5.5.1 Pseudocode

```

INPUT  $x_1$ 
 $minimum \leftarrow x_1$ 
 $maximum \leftarrow x_1$ 
FOR  $i \leftarrow 2$  TO  $N$ 
    INPUT  $x_i$ 
    IF  $x_i < minimum$ 
         $minimum \leftarrow x_i$ 
    END IF
    IF  $x_i > maximum$ 
         $maximum \leftarrow x_i$ 
    END IF
END FOR

```

Pendekatan tersebut bekerja untuk data positif maupun negatif. Kita tidak membuat asumsi bahwa nol selalu lebih kecil atau lebih besar daripada data yang akan diproses.

### 5.5.2 Mengapa tidak selalu menggunakan nol?

Misalkan seluruh data yang diproses bernilai negatif. Sebagai contoh, perhatikan data berikut.

-4, -2, -7.

Jika kita menulis `maximum = 0.0`, hasil maksimum akan tetap nol. Nilai tersebut salah karena nol bahkan tidak terdapat di dalam data.

Sebaliknya, jika seluruh data positif dan kita menulis `minimum = 0.0`, hasil minimum juga salah. Karena itu, menggunakan data pertama sebagai inisialisasi biasanya lebih aman.

## 5.6 Menggabungkan beberapa pola dalam satu iterasi

Satu pengulangan dapat memperbarui beberapa keadaan sekaligus. Hal ini sering lebih efisien daripada membaca data berkali-kali untuk tujuan yang berbeda.

Misalkan kita ingin memperoleh jumlah, rata-rata, minimum, maksimum, serta jumlah data di atas ambang. Semua informasi tersebut dapat diperbarui ketika setiap data dibaca satu kali.

### 5.6.1 Implementasi terpadu

```

#include <stdio.h>

int main(void)
{
    int n;
    double limit;

    printf("Number of measurements: ");
    scanf("%d", &n);

```

```
printf("Limit: ");
scanf("%lf", &limit);

if (n <= 0)
{
    printf("N must be positive.\n");
    return 1;
}

double x;

printf("x1: ");
scanf("%lf", &x);

double sum = x;
double minimum = x;
double maximum = x;
int count_high = (x > limit) ? 1 : 0;

for (int i = 2; i <= n; i++)
{
    printf("x%d: ", i);
    scanf("%lf", &x);

    sum += x;

    if (x < minimum)
    {
        minimum = x;
    }

    if (x > maximum)
    {
        maximum = x;
    }

    if (x > limit)
    {
        count_high++;
    }
}

double mean = sum / n;

printf("\nMean      = %.3f\n", mean);
printf("Minimum    = %.3f\n", minimum);
printf("Maximum     = %.3f\n", maximum);
printf("Above limit = %d\n", count_high);

return 0;
```

```
}

```

Program tersebut memperlihatkan bahwa pola iteratif dapat dikombinasikan dalam satu proses. Setiap data hanya perlu dibaca dan diproses satu kali.

## 5.7 Pola *sentinel*

Pada beberapa persoalan, jumlah data tidak diketahui sebelumnya. Proses dapat dihentikan ketika pengguna memasukkan nilai khusus yang disebut **sentinel**.

Misalkan temperatur dibaca terus tanpa mengetahui jumlah datanya terlebih dahulu. Proses akan dihentikan ketika pengguna memasukkan nilai khusus berikut.

-999.

Nilai -999 dianggap sebagai tanda berhenti dan tidak termasuk data. Pola *sentinel* tersebut dapat dinyatakan dengan pseudocode sederhana. Bentuknya dapat ditulis sebagai berikut.

```
sum ← 0
count ← 0
INPUT temperature
WHILE temperature ≠ -999
    sum ← sum + temperature
    count ← count + 1
    INPUT temperature
END WHILE
```

### 5.7.1 Implementasi dalam C

```
#include <stdio.h>

int main(void)
{
    const double sentinel = -999.0;

    double sum = 0.0;
    int count = 0;

    double temperature;

    printf("Temperature (-999 to stop): ");
    scanf("%lf", &temperature);

    while (temperature != sentinel)
    {
        sum += temperature;
        count++;

        printf("Temperature (-999 to stop): ");
        scanf("%lf", &temperature);
    }
}
```

```
if (count > 0)
{
    double mean = sum / count;

    printf("Count = %d\n", count);
    printf("Mean = %.3f C\n", mean);
}
else
{
    printf("No valid data.\n");
}

return 0;
}
```

Program tersebut menggunakan pola **priming read**, yaitu pembacaan pertama dilakukan sebelum masuk ke **while**. Setelah setiap data valid diproses, pembacaan berikutnya dilakukan di akhir iterasi.

### 5.7.2 Memilih sentinel

Sentinel harus dipilih agar mudah dibedakan dari data valid. Jika nilai sentinel masih mungkin muncul sebagai data sebenarnya, desain algoritma menjadi ambigu.

Untuk temperatur Celsius, **-999** mungkin cukup aman dalam banyak konteks laboratorium sederhana. Namun, dalam sistem nyata, format input yang lebih eksplisit sering lebih baik daripada bergantung pada satu angka khusus.

## 5.8 Pola validasi masukan

Validasi masukan bertujuan memastikan data memenuhi syarat sebelum digunakan dalam perhitungan. Pola ini penting karena program teknik sering bekerja dengan besaran yang memiliki batas fisik atau batas operasional.

Misalkan efisiensi hanya boleh berada pada rentang fisik tertentu. Rentang valid yang digunakan dinyatakan sebagai berikut.

$$0 \leq \eta \leq 1.$$

Program harus menolak nilai yang berada di luar rentang tersebut. Dua contoh nilai yang tidak valid ditunjukkan berikut ini.

$$\eta = -0.2$$

atau

$$\eta = 1.4.$$

### 5.8.1 Pseudocode

```

DO
  INPUT  $\eta$ 
  IF  $\eta < 0$  OR  $\eta > 1$ 
    OUTPUT "invalid"
  END IF
WHILE  $\eta < 0$  OR  $\eta > 1$ 

```

### 5.8.2 Implementasi

```

#include <stdio.h>

int main(void)
{
    double efficiency;

    do
    {
        printf("Efficiency [0, 1]: ");
        scanf("%lf", &efficiency);

        if (efficiency < 0.0 ||
            efficiency > 1.0)
        {
            printf("Invalid efficiency.\n");
        }
    }
    while (efficiency < 0.0 ||
           efficiency > 1.0);

    printf("Accepted value = %.3f\n",
           efficiency);

    return 0;
}

```

Pola ini memisahkan dua gagasan, yaitu memperoleh input dan memastikan input memenuhi syarat. Pengulangan berhenti hanya jika kondisi valid tercapai.

## 5.9 Validasi berdasarkan batas fisik

Validasi tidak hanya berkaitan dengan format input. Kita juga dapat memeriksa apakah nilai masuk akal secara fisis.

Misalkan massa harus bernilai positif secara fisik. Syarat validitasnya dapat ditulis sebagai berikut.

$$m > 0.$$

Program dapat menggunakan

```
while (mass <= 0.0)
{
    printf("Mass must be positive.\n");
    printf("Mass (kg): ");
    scanf("%lf", &mass);
}
```

Jika tekanan absolut digunakan, nilai negatif juga tidak masuk akal secara fisika. Batas valid selalu harus disesuaikan dengan model dan definisi besaran yang digunakan.

### 5.10 Validasi input dari `scanf`

Selain memeriksa rentang nilai, kita juga perlu memeriksa apakah `scanf` berhasil membaca tipe data yang diharapkan. Fungsi `scanf` mengembalikan jumlah item yang berhasil dikonversi.

Contoh:

```
double x;

if (scanf("%lf", &x) != 1)
{
    printf("Invalid input.\n");
    return 1;
}
```

Untuk program sederhana, penghentian langsung seperti contoh di atas sudah cukup sebagai pengantar. Untuk program yang lebih kuat, input dapat dibersihkan lalu pengguna diminta mencoba kembali.

### 5.11 Pengulangan bersarang

**Nested loop** adalah pengulangan yang berada di dalam pengulangan lain. Pola ini sering muncul ketika kita bekerja dengan dua indeks, tabel dua dimensi, kombinasi pasangan, atau eksperimen parametrik.

Contoh sederhana:

```
for (int i = 1; i <= 3; i++)
{
    for (int j = 1; j <= 4; j++)
    {
        printf("i = %d, j = %d\n", i, j);
    }
}
```

Pengulangan luar berjalan tiga kali, sedangkan pengulangan dalam berjalan empat kali untuk setiap nilai `i`. Dengan demikian, jumlah kombinasi yang diproses dapat dihitung dari perkalian kedua jumlah iterasi.

$$3 \times 4 = 12.$$

### 5.11.1 Kompleksitas

Jika batas kedua pengulangan sama-sama sebesar  $N$ , jumlah operasi bertumbuh jauh lebih cepat daripada satu loop. Secara kasar, pola pertumbuhannya dapat dinyatakan melalui hubungan berikut.

$$N^2.$$

Pola pertumbuhan tersebut memiliki nama khusus dalam analisis algoritma. Oleh karena itu, pengulangan bersarang seperti ini sering dikaitkan dengan kompleksitas berikut.

$$O(N^2).$$

Kita tetap harus berhati-hati karena tidak semua nested loop otomatis  $O(N^2)$ . Kompleksitas yang tepat bergantung pada batas masing-masing pengulangan.

## 5.12 Tabel komputasi

Pengulangan sangat cocok digunakan untuk membuat tabel nilai suatu model fisika. Tabel tersebut dapat digunakan untuk memeriksa perilaku model, membuat grafik, atau menjadi data awal untuk analisis lebih lanjut.

Misalkan kita ingin mengevaluasi model gerak lurus dengan percepatan konstan. Model matematis yang digunakan dituliskan sebagai berikut.

$$x(t) = x_0 + v_0 t + \frac{1}{2} a t^2.$$

Model tersebut akan dievaluasi pada beberapa titik waktu diskrit. Nilai waktu yang digunakan dapat dituliskan sebagai berikut.

$$t = 0, 1, 2, \dots, 10.$$

### 5.12.1 Implementasi

```
#include <stdio.h>

int main(void)
{
    const double x0 = 0.0;
    const double v0 = 2.0;
    const double acceleration = 1.5;

    printf("t(s)\tx(m)\n");

    for (int t = 0; t <= 10; t++)
    {
        double position =
            x0
            + v0 * t
            + 0.5 * acceleration * t * t;

        printf("%d\t%.3f\n", t, position);
    }
}
```

```
    }  
  
    return 0;  
}
```

Keluaran program membentuk tabel yang dapat diperiksa secara manual. Pada kuliah tentang pengolahan berkas, pola yang sama dapat dikembangkan untuk menyimpan hasil ke file.

### 5.13 Tabel dengan langkah real

Tidak semua parameter berubah dalam langkah integer. Dalam komputasi fisika, kita sering ingin mengevaluasi model pada langkah real. Sebagai contoh, kita dapat menggunakan langkah waktu

$$\Delta t = 0.1 \text{ s.}$$

Salah satu pendekatan yang cukup aman adalah menggunakan pencacah integer lalu menghitung nilai real dari pencacah tersebut. Pendekatan ini mengurangi ketergantungan pada akumulasi pembulatan *floating-point*.

Contoh:

```
#include <stdio.h>  
  
int main(void)  
{  
    const double dt = 0.1;  
  
    for (int i = 0; i <= 100; i++)  
    {  
        double t = i * dt;  
  
        printf("%.1f\n", t);  
    }  
  
    return 0;  
}
```

Alternatif seperti `for (double t = 0.0; t <= 10.0; t += 0.1)` dapat digunakan, tetapi nilai `t` mengalami pembulatan berulang. Untuk pengantar komputasi ilmiah, pencacah integer sering lebih mudah dikendalikan.

### 5.14 Studi kasus: Hukum Hooke

Hukum Hooke memberikan hubungan linear antara gaya pegas dan perpindahan. Untuk pegas ideal, hubungan tersebut dituliskan sebagai berikut.

$$F = -kx.$$

Kita ingin mengevaluasi gaya pada beberapa nilai perpindahan. Rentang perpindahan yang digunakan dituliskan sebagai berikut.

$$x = -0.10, -0.09, \dots, 0.10 \text{ m.}$$

Kita dapat menggunakan pencacah integer dari  $-10$  sampai  $10$ , kemudian mengubahnya menjadi nilai perpindahan. Pendekatan tersebut membuat langkah  $0.01$  mudah dikontrol.

```
#include <stdio.h>

int main(void)
{
    const double k = 100.0;

    printf("x(m)\tF(N)\n");

    for (int i = -10; i <= 10; i++)
    {
        double x = i * 0.01;
        double force = -k * x;

        printf("%.2f\t%.3f\n", x, force);
    }

    return 0;
}
```

Program tersebut menghasilkan data yang simetris terhadap titik  $x = 0$ . Pola fisik tersebut dapat digunakan sebagai salah satu cara memeriksa kewajaran keluaran.

## 5.15 Studi kasus: pemetaan parameter dua dimensi

Nested loop dapat digunakan untuk mengevaluasi model pada dua parameter. Sebagai contoh, kita dapat memvariasikan tegangan dan arus secara bersamaan untuk menghitung daya listrik. Hubungan yang digunakan adalah

$$P = VI$$

Perhitungan dilakukan untuk beberapa kombinasi tegangan dan arus. Dengan demikian, setiap pasangan parameter menghasilkan satu nilai daya.

Misalkan

$$V = 5, 10, 15 \text{ V}$$

dan

$$I = 1, 2, 3, 4 \text{ A.}$$

### 5.15.1 Implementasi

```
#include <stdio.h>
```

```
int main(void)
{
    for (int voltage = 5;
        voltage <= 15;
        voltage += 5)
    {
        for (int current = 1;
            current <= 4;
            current++)
        {
            double power =
                voltage * current;

            printf("V = %2d V, I = %d A, "
                "P = %.1f W\n",
                voltage, current, power);
        }

        printf("\n");
    }

    return 0;
}
```

Program tersebut menghitung seluruh kombinasi pasangan tegangan dan arus. Jumlah kombinasi sama dengan hasil perkalian banyak nilai pada kedua parameter.

## 5.16 Pola pembaruan iteratif

Tidak semua iterasi hanya membaca data. Beberapa algoritma menggunakan nilai hasil iterasi sebelumnya untuk menghasilkan nilai baru.

Secara abstrak,

$$x_{k+1} = f(x_k).$$

Kita memulai dari tebakan awal  $x_0$  dan menghasilkan urutan  $x_1, x_2, x_3, \dots$ . Proses dihentikan ketika suatu kriteria penghentian terpenuhi.

Pola tersebut sangat penting dalam metode numerik, optimasi, simulasi, dan pemodelan dinamik. Pada kuliah ini kita hanya membahas bentuk sederhana agar struktur algoritmanya dapat dipahami dengan baik.

## 5.17 Kriteria penghentian

Algoritma iteratif harus memiliki kondisi yang menentukan kapan proses berhenti. Kondisi ini disebut **kriteria penghentian** (*stopping criterion*).

Kriteria penghentian dapat dirumuskan dengan beberapa cara. Daftar berikut menunjukkan beberapa bentuk yang umum digunakan.

- ▶ jumlah iterasi mencapai batas maksimum;

- ▶ perubahan antariterasi cukup kecil;
- ▶ galat terhadap target cukup kecil;
- ▶ suatu kondisi fisik tercapai.

Misalkan

$$|x_{k+1} - x_k| < \varepsilon.$$

Kita dapat menggunakan kondisi tersebut untuk menyatakan bahwa perubahan hasil sudah cukup kecil. Nilai  $\varepsilon$  disebut toleransi dan harus dipilih sesuai skala serta tujuan perhitungan.

### 5.17.1 Mengapa batas iterasi maksimum diperlukan?

Kondisi berbasis toleransi saja tidak selalu menjamin algoritma berhenti. Jika metode tidak konvergen, program dapat berjalan terus tanpa mencapai kondisi yang diinginkan.

Karena itu, algoritma numerik yang baik biasanya juga menggunakan batas `maximum_iteration` sebagai perlindungan tambahan. Kombinasi toleransi dan batas iterasi membuat program lebih aman serta lebih mudah didiagnosis.

## 5.18 Contoh iteratif sederhana: pendekatan akar kuadrat

Kita ingin mencari nilai  $x$  yang memenuhi suatu persamaan sederhana. Persamaan targetnya dapat dituliskan sebagai berikut.

$$x^2 = 2.$$

Salah satu pendekatan paling sederhana adalah memulai dari nol lalu menaikkan  $x$  dengan langkah kecil. Metode ini tidak efisien, tetapi struktur iteratifnya mudah dipahami.

### 5.18.1 Algoritma

```
x ← 0
dx ← 0.001
WHILE x · x < 2
  x ← x + dx
END WHILE
OUTPUT x
```

### 5.18.2 Implementasi

```
#include <stdio.h>

int main(void)
{
    const double target = 2.0;
    const double dx = 0.001;

    double x = 0.0;
```

```

while (x * x < target)
{
    x += dx;
}

printf("Approximation = %.6f\n", x);

return 0;
}

```

Hasil pendekatan dapat dibandingkan dengan nilai referensi yang sudah diketahui. Nilai referensi yang digunakan dituliskan sebagai berikut.

$$\sqrt{2} \approx 1.41421356.$$

Galat pendekatan bergantung pada ukuran langkah  $dx$ . Langkah yang lebih kecil biasanya meningkatkan ketelitian, tetapi membutuhkan lebih banyak iterasi.

### 5.19 Iterasi berdasarkan perubahan hasil

Kita dapat menggunakan perubahan antariterasi sebagai kriteria berhenti. Struktur umumnya dapat ditulis dengan membandingkan nilai lama dan nilai baru.

```

old_value ← tebakan awal
new_value ← pembaruan(old_value)
WHILE |new_value - old_value| > tolerance
    old_value ← new_value
    new_value ← pembaruan(old_value)
END WHILE

```

Pola tersebut akan banyak muncul dalam komputasi numerik. Kita belum membahas metode tertentu secara mendalam, tetapi penting untuk mengenali struktur umumnya.

### 5.20 Contoh: metode Babilonia untuk akar kuadrat

Ada metode iteratif yang jauh lebih efisien daripada langkah tetap. Salah satu algoritma klasik untuk mendekati  $\sqrt{S}$  adalah iterasi

$$x_{k+1} = \frac{1}{2} \left( x_k + \frac{S}{x_k} \right).$$

Untuk  $S > 0$ , kita dapat memilih tebakan awal positif dan memperbarui nilai sampai perubahan antariterasi cukup kecil. Metode ini biasanya jauh lebih cepat daripada menaikkan nilai dengan langkah tetap.

#### 5.20.1 Implementasi

```

#include <stdio.h>
#include <math.h>

```

```
int main(void)
{
    const double s = 2.0;
    const double tolerance = 1e-10;
    const int max_iteration = 100;

    double x = 1.0;
    int iteration = 0;

    while (iteration < max_iteration)
    {
        double x_new =
            0.5 * (x + s / x);

        double difference =
            fabs(x_new - x);

        x = x_new;
        iteration++;

        if (difference < tolerance)
        {
            break;
        }
    }

    printf("sqrt(%.3f) ~= %.12f\n", s, x);
    printf("Iterations = %d\n", iteration);

    return 0;
}
```

Contoh ini menunjukkan pola iterasi yang lebih realistis. Program menggunakan tebakan awal, pembaruan keadaan, toleransi, serta batas iterasi maksimum.

### 5.20.2 Memeriksa konvergensi

Kita dapat menampilkan setiap iterasi untuk melihat bagaimana nilai berubah. Teknik ini sangat berguna ketika mempelajari perilaku algoritma numerik dan mencari kesalahan pembaruan.

Tambahkan

```
printf("%d %.12f\n",
    iteration, x);
```

di dalam pengulangan. Dari keluaran tersebut kita dapat melihat apakah urutan nilai mendekati suatu nilai tetap.

## 5.21 Pola konvergensi deret

Pengulangan juga digunakan untuk menghitung jumlah parsial suatu deret. Contoh yang sederhana adalah deret kuadrat terbalik. Bentuk jumlah parsialnya adalah

$$S_N = \sum_{n=1}^N \frac{1}{n^2}.$$

Nilai  $S_N$  mendekati

$$\frac{\pi^2}{6}$$

ketika  $N$  semakin besar. Kita dapat menggunakan pola akumulator untuk mengamati konvergensi tersebut.

```
#include <stdio.h>

int main(void)
{
    const double pi =
        3.141592653589793;

    const double reference =
        pi * pi / 6.0;

    int n_max;

    printf("N: ");
    scanf("%d", &n_max);

    if (n_max <= 0)
    {
        printf("N must be positive.\n");
        return 1;
    }

    double sum = 0.0;

    for (int n = 1; n <= n_max; n++)
    {
        double nd = (double)n;
        sum += 1.0 / (nd * nd);
    }

    double error = reference - sum;

    printf("S_N      = %.12f\n", sum);
    printf("Reference = %.12f\n", reference);
    printf("Error     = %.12e\n", error);

    return 0;
}
```

Program tersebut menghubungkan pola akumulator dengan gagasan konvergensi numerik. Kita dapat menjalankannya untuk beberapa nilai  $N$  dan mengamati bagaimana galat berubah.

## 5.22 Menghentikan deret berdasarkan ukuran suku

Alih-alih menentukan  $N$  terlebih dahulu, kita dapat menghentikan proses ketika suku berikutnya sudah sangat kecil. Strategi ini merupakan contoh pengulangan berbasis kondisi.

Kita juga dapat menghentikan proses berdasarkan ukuran suku, bukan berdasarkan jumlah iterasi. Misalkan kriteria penghentian yang digunakan dinyatakan sebagai berikut.

$$\frac{1}{n^2} < 10^{-6}.$$

Pseudocode:

```
n ← 1
sum ← 0
term ← 1/n2
WHILE term ≥ tolerance
    sum ← sum + term
    n ← n + 1
    term ← 1/n2
END WHILE
```

Pola tersebut memperlihatkan bahwa kondisi berhenti dapat diturunkan dari sifat matematis masalah. Namun, ukuran suku kecil belum selalu berarti galat total kecil untuk semua jenis deret, sehingga interpretasi harus disesuaikan dengan teori yang relevan.

## 5.23 Pola *sentinel* dan validasi sekaligus

Kadang-kadang kita perlu membedakan tiga keadaan input, yaitu data valid, data tidak valid, dan sentinel untuk berhenti. Urutan pemeriksaan harus dirancang dengan jelas agar ketiganya tidak tercampur.

Misalkan kita mempunyai rentang temperatur yang dianggap valid secara operasional. Rentang valid tersebut dinyatakan sebagai berikut.

$$-100^{\circ}\text{C} \leq T \leq 200^{\circ}\text{C},$$

sedangkan sentinel adalah  $-999$ .

Algoritma harus memeriksa sentinel terlebih dahulu. Jika bukan sentinel, barulah nilai diperiksa apakah berada pada rentang valid.

### 5.23.1 Implementasi

```
#include <stdio.h>

int main(void)
```

```
{
    const double sentinel = -999.0;

    double sum = 0.0;
    int count = 0;

    while (1)
    {
        double temperature;

        printf("Temperature (-999 to stop): ");
        scanf("%lf", &temperature);

        if (temperature == sentinel)
        {
            break;
        }

        if (temperature < -100.0 ||
            temperature > 200.0)
        {
            printf("Invalid measurement.\n");
            continue;
        }

        sum += temperature;
        count++;
    }

    if (count > 0)
    {
        printf("Mean = %.3f C\n",
            sum / count);
    }
    else
    {
        printf("No valid measurement.\n");
    }

    return 0;
}
```

Program tersebut menggabungkan **break**, **continue**, sentinel, validasi, counter, dan akumulator. Walaupun strukturnya lebih kompleks, setiap bagian memiliki peran yang jelas.

## 5.24 Studi kasus terpadu: analisis getaran mesin

Sekarang kita merancang program untuk membaca data percepatan getaran dari sebuah akselerometer yang dipasang pada mesin. Kasus ini menarik karena sinyal getaran dapat

bernilai positif maupun negatif, sehingga rata-rata saja tidak cukup untuk menggambarkan besar getaran.

Selain rata-rata, kita ternyata perlu menghitung nilai RMS (*root mean square*) sebagai ukuran sederhana besar getaran. Nilai RMS didefinisikan sebagai

$$a_{\text{RMS}} = \sqrt{\frac{1}{N} \sum_{i=1}^N a_i^2}.$$

Besaran RMS selalu tidak negatif dan sensitif terhadap amplitudo sinyal. Karena itu, RMS sering lebih informatif daripada rata-rata untuk data getaran yang berosilasi di sekitar nol.

Program akan membaca data sampai sentinel diberikan. Data yang berada di luar rentang fisik yang kita tetapkan akan dianggap tidak valid dan tidak ikut dihitung.

Kita ingin memperoleh

- ▶ jumlah data valid;
- ▶ rata-rata percepatan;
- ▶ nilai RMS;
- ▶ minimum;
- ▶ maksimum;
- ▶ jumlah data dengan amplitudo di atas ambang;
- ▶ persentase data dengan amplitudo di atas ambang.

### 5.24.1 Spesifikasi

Sentinel digunakan untuk menandai akhir masukan. Nilai sentinel dipilih jauh di luar rentang data valid.

$$a_{\text{sentinel}} = 9999.$$

Rentang percepatan yang dianggap valid adalah

$$-50 \leq a \leq 50 \quad \text{m/s}^2.$$

Kita juga menetapkan ambang amplitudo getaran

$$a_{\text{limit}} = 8 \text{ m/s}^2.$$

Suatu data dianggap melampaui ambang jika

$$|a_i| > a_{\text{limit}}.$$

### 5.24.2 Algoritma

Kita memerlukan dua akumulator, yaitu `sum` dan `sum_square`. Akumulator kedua menyimpan jumlah kuadrat data untuk menghitung RMS.

```

sum ← 0
sum_square ← 0
count ← 0
count_high ← 0
WHILE true
    INPUT a
    IF a = sentinel
        BREAK
    END IF
    IF a < -50 OR a > 50
        OUTPUT "invalid"
        CONTINUE
    END IF
    IF count = 0
        minimum ← a
        maximum ← a
    ELSE
        IF a < minimum
            minimum ← a
        END IF
        IF a > maximum
            maximum ← a
        END IF
    END IF
    sum ← sum + a
    sum_square ← sum_square + a · a
    count ← count + 1
    IF |a| > alimit
        count_high ← count_high + 1
    END IF
END WHILE

```

Setelah proses selesai, rata-rata dan RMS hanya dihitung jika terdapat setidaknya satu data valid. Pemeriksaan ini tetap diperlukan karena pengguna dapat langsung memasukkan sentinel atau hanya memberikan data tidak valid.

### 5.24.3 Implementasi C

```

#include <stdio.h>
#include <math.h>

int main(void)
{
    const double sentinel = 9999.0;
    const double min_valid = -50.0;
    const double max_valid = 50.0;
    const double limit = 8.0;

    double sum = 0.0;
    double sum_square = 0.0;

```

```
int count = 0;
int count_high = 0;

double minimum = 0.0;
double maximum = 0.0;

while (1)
{
    double acceleration;

    printf("Acceleration (9999 to stop): ");
    scanf("%lf", &acceleration);

    if (acceleration == sentinel)
    {
        break;
    }

    if (acceleration < min_valid ||
        acceleration > max_valid)
    {
        printf("Invalid measurement.\n");
        continue;
    }

    if (count == 0)
    {
        minimum = acceleration;
        maximum = acceleration;
    }
    else
    {
        if (acceleration < minimum)
        {
            minimum = acceleration;
        }

        if (acceleration > maximum)
        {
            maximum = acceleration;
        }
    }

    sum += acceleration;

    sum_square +=
        acceleration * acceleration;

    count++;
}
```

```

        if (fabs(acceleration) > limit)
        {
            count_high++;
        }
    }

    if (count == 0)
    {
        printf("No valid measurement.\n");
        return 0;
    }

    double mean =
        sum / count;

    double rms =
        sqrt(sum_square / count);

    double percentage_high =
        100.0 * count_high / count;

    printf("\nValid data      = %d\n", count);
    printf("Mean           = %.3f m/s^2\n", mean);
    printf("RMS             = %.3f m/s^2\n", rms);
    printf("Minimum          = %.3f m/s^2\n", minimum);
    printf("Maximum           = %.3f m/s^2\n", maximum);
    printf("Above limit      = %d\n", count_high);
    printf("Percentage        = %.2f %%\n",
        percentage_high);

    return 0;
}

```

Program tersebut menggabungkan beberapa pola iteratif dalam satu kali pembacaan data. Selain `sum`, kita menggunakan `sum_square` sehingga nilai RMS dapat dihitung tanpa menyimpan seluruh data di dalam larik.

#### 5.24.4 Mengapa RMS berguna?

Data getaran sering memiliki nilai positif dan negatif karena arah percepatan berubah selama osilasi. Jika sinyal berosilasi hampir simetris terhadap nol, rata-ratanya dapat kecil walaupun amplitudo getarannya cukup besar.

Sebagai contoh, pasangan nilai +10, -10 memiliki rata-rata nol. Namun, RMS-nya adalah

$$a_{\text{RMS}} = \sqrt{\frac{10^2 + (-10)^2}{2}} = 10.$$

Contoh ini menunjukkan bahwa rata-rata dan RMS membawa informasi yang berbeda. Rata-rata menggambarkan kecenderungan nilai bertanda, sedangkan RMS memberikan ukuran amplitudo efektif.

5.25 Tracing studi kasus

Misalkan data percepatan yang dibaca adalah

1.2, -2.4, 8.7, 75, -9.1, 3.6, 9999.

Nilai 75 ditolak karena berada di luar rentang valid. Sentinel 9999 menghentikan proses dan tidak diperlakukan sebagai data pengukuran.

Tracing keadaan utama dapat dituliskan dalam tabel berikut.

| Input | Valid?   | count | sum  | sum_square | minimum | maximum | count_high |
|-------|----------|-------|------|------------|---------|---------|------------|
| 1.2   | ya       | 1     | 1.2  | 1.44       | 1.2     | 1.2     | 0          |
| -2.4  | ya       | 2     | -1.2 | 7.20       | -2.4    | 1.2     | 0          |
| 8.7   | ya       | 3     | 7.5  | 82.89      | -2.4    | 8.7     | 1          |
| 75    | tidak    | 3     | 7.5  | 82.89      | -2.4    | 8.7     | 1          |
| -9.1  | ya       | 4     | -1.6 | 165.70     | -9.1    | 8.7     | 2          |
| 3.6   | ya       | 5     | 2.0  | 178.66     | -9.1    | 8.7     | 2          |
| 9999  | sentinel | 5     | 2.0  | 178.66     | -9.1    | 8.7     | 2          |

Rata-rata percepatan adalah

$$\bar{a} = \frac{2.0}{5} = 0.4 \text{ m/s}^2.$$

Nilai RMS dihitung sebagai

$$a_{\text{RMS}} = \sqrt{\frac{178.66}{5}} \approx 5.978 \text{ m/s}^2.$$

Terdapat dua data yang memenuhi

$$|a_i| > 8 \text{ m/s}^2.$$

Dengan demikian, persentase data yang melampaui ambang adalah

$$\frac{2}{5} \times 100\% = 40\%.$$

Hasil tersebut menunjukkan perbedaan yang cukup jelas antara rata-rata dan RMS. Walaupun rata-rata hanya 0.4 m/s<sup>2</sup>, nilai RMS hampir 6 m/s<sup>2</sup> karena data memiliki osilasi positif dan negatif yang cukup besar.

5.26 Kesalahan umum pada algoritma iteratif

5.26.1 Inisialisasi yang tidak sesuai

Kesalahan inisialisasi dapat merusak hasil sejak iterasi pertama. Contohnya adalah menggunakan maximum = 0 ketika semua data mungkin bernilai negatif.

Selalu tanyakan makna variabel sebelum iterasi dimulai. Nilai awal harus konsisten dengan invarian yang ingin dipertahankan.

### 5.26.2 Lupa memperbarui keadaan

Pengulangan dapat menjadi tidak berhenti jika variabel kondisi tidak pernah berubah. Masalah ini umum terjadi pada `while`.

Contoh salah:

```
int i = 0;

while (i < 10)
{
    printf("%d\n", i);
}
```

Nilai `i` tidak pernah diperbarui. Oleh karena itu, kondisi `i < 10` selalu benar.

### 5.26.3 Pembaruan dilakukan pada posisi yang salah

Urutan pembaruan dapat memengaruhi hasil. Dalam algoritma tertentu, memperbarui variabel terlalu awal dapat membuat nilai lama hilang sebelum digunakan.

Karena itu, tracing beberapa iterasi pertama sangat berguna. Kita dapat melihat apakah setiap variabel berubah pada saat yang tepat.

### 5.26.4 Sentinel ikut dihitung

Kesalahan umum pada sentinel adalah memproses nilai sentinel sebelum memeriksa apakah nilai tersebut menandakan berhenti. Akibatnya, sentinel dapat masuk ke dalam `sum`, `count`, minimum, atau maksimum.

Pemeriksaan sentinel sebaiknya dilakukan sebelum data diproses. Hal ini membuat makna sentinel tetap konsisten sebagai sinyal, bukan data.

### 5.26.5 Membagi dengan nol

Pada algoritma dengan jumlah data tidak diketahui, bisa saja pengguna langsung memasukkan sentinel. Dalam kasus itu, nilai `count` tetap nol.

Rata-rata tidak boleh dihitung secara langsung ketika belum ada data valid. Kondisi yang harus dipenuhi sebelum pembagian dilakukan dituliskan sebagai berikut.

$$\text{count} > 0.$$

Pemeriksaan kasus kosong harus menjadi bagian dari desain algoritma, bukan tambahan setelah program gagal. Kasus batas semacam ini sering lebih mudah ditemukan jika kita melakukan tracing sebelum menulis kode.

### 5.26.6 Kriteria berhenti tidak realistis

Pada algoritma numerik, kondisi seperti `while (x != target)` sering tidak aman untuk `double`. Representasi *floating-point* dapat membuat nilai tidak pernah sama persis dengan target.

Gunakan toleransi seperti

$$|x - \text{target}| < \varepsilon$$

jika konteks numeriknya memang sesuai. Batas iterasi maksimum juga sebaiknya disediakan agar program tetap mempunyai jalan keluar ketika metode tidak konvergen.

### 5.26.7 Menggunakan langkah yang terlalu besar atau terlalu kecil

Pada pencarian iteratif berbasis langkah tetap, ukuran langkah memengaruhi ketelitian dan biaya komputasi. Langkah terlalu besar dapat melewati solusi yang diinginkan, sedangkan langkah terlalu kecil dapat membutuhkan sangat banyak iterasi.

Tidak ada ukuran langkah universal yang selalu terbaik. Pilihannya harus disesuaikan dengan skala persoalan dan tujuan komputasi.

## 5.27 Efisiensi pola iteratif

Pola counter, akumulator, minimum, maksimum, dan validasi biasanya membutuhkan pekerjaan konstan untuk setiap data. Jika terdapat  $N$  data, jumlah pekerjaan bertumbuh sebanding dengan  $N$ . Kompleksitas waktunya biasanya  $O(N)$ .

Kita tidak selalu harus menyimpan seluruh data untuk memperoleh statistik sederhana. Dalam pemrosesan satu per satu, kebutuhan memori tambahan dapat tetap pada orde berikut.

$$O(1)$$

jika data diproses satu per satu tanpa disimpan seluruhnya. Pendekatan ini disebut pemrosesan secara *streaming*.

Nested loop dapat menghasilkan kompleksitas lebih tinggi. Jika dua pengulangan masing-masing berjalan sekitar  $N$  kali, jumlah kombinasi bertumbuh secara kuadratik. Kompleksitasnya dapat menjadi  $O(N^2)$ .

Analisis kompleksitas membantu kita memahami apakah pola yang dipilih masih masuk akal ketika ukuran data membesar. Namun, kebenaran algoritma tetap menjadi syarat pertama sebelum efisiensi dianalisis.

## 5.28 Kebiasaan merancang algoritma iteratif

Sebelum menulis kode, tentukan variabel keadaan yang benar-benar diperlukan. Tuliskan makna setiap variabel agar proses pembaruan tidak dilakukan secara mekanis.

Tentukan nilai awal setiap variabel dan jelaskan mengapa nilai tersebut benar. Untuk minimum dan maksimum, pertimbangkan apakah data pertama lebih sesuai daripada konstanta tertentu.

Tuliskan kondisi penghentian secara eksplisit. Pastikan setiap iterasi membawa keadaan lebih dekat ke kondisi berhenti atau memiliki batas iterasi maksimum.

Lakukan tracing menggunakan data kecil. Gunakan kasus biasa, kasus batas, data negatif, data kosong, serta input yang tidak valid jika relevan.

Periksa hasil berdasarkan matematika dan fisika. Program yang berjalan tanpa *error* tetap dapat menghasilkan nilai yang tidak masuk akal.

### 5.29 Cek pemahaman

Cobalah menjawab pertanyaan berikut tanpa melihat kembali catatan. Setelah menjawab, bandingkan kembali jawaban Anda dengan pola algoritma yang telah dibahas.

- ▶ Apa yang dimaksud dengan keadaan (*state*) dalam algoritma iteratif?
- ▶ Apa tiga komponen utama suatu iterasi?
- ▶ Apa fungsi *counter*?
- ▶ Apa fungsi akumulator/*accumulator*?
- ▶ Mengapa akumulator penjumlahan biasanya dimulai dari nol?
- ▶ Mengapa minimum dan maksimum sebaiknya diinisialisasi menggunakan data pertama?
- ▶ Apa perbedaan antara kasus jumlah data diketahui dan tidak diketahui?
- ▶ Apa yang dimaksud dengan sentinel?
- ▶ Mengapa sentinel tidak boleh ikut diproses sebagai data?
- ▶ Apa yang dimaksud dengan *priming read*?
- ▶ Bagaimana validasi input berbeda dari sentinel?
- ▶ Mengapa nested loop dapat menghasilkan kompleksitas  $O(N^2)$ ?
- ▶ Mengapa pencacah integer sering lebih aman untuk membuat langkah real tetap?
- ▶ Apa yang dimaksud dengan pembaruan iteratif?
- ▶ Apa fungsi toleransi pada algoritma numerik?
- ▶ Mengapa batas iterasi maksimum tetap diperlukan walaupun toleransi sudah digunakan?
- ▶ Apa risiko menggunakan `x != target` pada nilai *floating-point*?
- ▶ Bagaimana tracing membantu menemukan kesalahan iterasi?
- ▶ Apa yang dimaksud dengan pemrosesan *streaming*?
- ▶ Mengapa counter, akumulator, minimum, dan maksimum dapat diperbarui dalam satu loop?
- ▶ Apa hubungan pola iteratif dengan konsep invarian?
- ▶ Mengapa program harus memeriksa `count > 0` sebelum menghitung rata-rata?
- ▶ Apa pengaruh ukuran langkah terhadap metode iteratif sederhana?
- ▶ Mengapa algoritma yang benar tetap perlu diperiksa secara fisika?

### 5.30 Latihan

### 5.30.1 Analisis data arus listrik secara *streaming*

Sebuah sistem akuisisi membaca arus listrik satu per satu tanpa mengetahui jumlah data sebelumnya. Input dihentikan menggunakan sentinel **-999**, sedangkan data valid harus berada pada rentang

$$-20 \leq I \leq 20 \text{ A.}$$

Buat algoritma dan program C yang dalam satu kali penelusuran menghitung jumlah data valid, rata-rata, minimum, maksimum, RMS, serta jumlah data yang memenuhi

$$|I| > 10 \text{ A.}$$

Program tidak boleh menyimpan seluruh data dalam larik. Jelaskan mengapa kebutuhan memori tambahannya tetap  $O(1)$  dan lakukan tracing menggunakan sekurang-kurangnya enam input yang mencakup data valid, data tidak valid, dan sentinel.

### 5.30.2 Statistik getaran dengan dua ambang

Sebuah akselerometer menghasilkan data percepatan  $a_i$ . Gunakan dua ambang,

$$a_{\text{warning}} = 5 \text{ m/s}^2$$

dan

$$a_{\text{danger}} = 10 \text{ m/s}^2.$$

Buat program yang mengklasifikasikan amplitudo setiap data berdasarkan  $|a_i|$  menjadi **NORMAL**, **WARNING**, atau **DANGER**. Dalam satu loop, hitung jumlah data pada setiap kategori, RMS keseluruhan, dan persentase kategori **DANGER**.

### 5.30.3 Studi konvergensi deret

Hitung

$$S_N = \sum_{n=1}^N \frac{1}{n^2}$$

untuk

$$N = 10, \quad 100, \quad 1000, \quad 10000.$$

Bandingkan setiap hasil dengan

$$S_{\infty} = \frac{\pi^2}{6}.$$

Buat tabel yang berisi  $N$ ,  $S_N$ , dan galat absolut. Setelah itu, modifikasi program agar berhenti berdasarkan ukuran suku berikutnya, lalu jelaskan perbedaan antara penghentian berdasarkan  $N$  dan penghentian berdasarkan toleransi.

### 5.30.4 Pengaruh ukuran langkah pada pencarian akar

Cari pendekatan untuk  $\sqrt{7}$  dengan metode langkah tetap. Lakukan eksperimen menggunakan

$$\Delta x = 0.1, \quad 0.01, \quad 0.001, \quad 0.0001.$$

Untuk setiap nilai  $\Delta x$ , catat hasil pendekatan, galat absolut terhadap nilai referensi, dan jumlah iterasi. Jelaskan hubungan antara ukuran langkah, ketelitian, dan biaya komputasi berdasarkan data yang diperoleh.

### 5.30.5 Metode Babilonia dengan perlindungan iterasi

Implementasikan metode Babilonia

$$x_{k+1} = \frac{1}{2} \left( x_k + \frac{S}{x_k} \right)$$

untuk menghitung  $\sqrt{S}$ . Program harus menerima  $S > 0$ , menggunakan toleransi yang diberikan pengguna, dan memiliki batas maksimum iterasi agar program tetap berhenti jika kondisi konvergensi gagal tercapai.

Tampilkan nomor iterasi,  $x_k$ ,  $x_{k+1}$ , dan  $|x_{k+1} - x_k|$  pada setiap langkah. Uji untuk sekurang-kurangnya tiga nilai  $S$  dan analisis seberapa cepat algoritma konvergen.

### 5.30.6 Pemetaan daya pada dua parameter

Gunakan dua pengulangan bersarang untuk menghitung

$$P = VI$$

pada sekumpulan nilai tegangan dan arus. Tegangan berubah dari 5 sampai 30 V dengan langkah 5 V, sedangkan arus berubah dari 0.5 sampai 5.0 A dengan langkah 0.5 A.

Selain mencetak tabel, cari kombinasi yang menghasilkan daya maksimum tanpa melebihi

$$P_{\max} = 100 \text{ W.}$$

Jelaskan bagaimana minimum-maksimum dan nested loop digabungkan pada persoalan ini. Tentukan pula banyak kombinasi parameter yang diperiksa program.

### 5.30.7 Validasi berlapis pada data sensor tekanan

Sebuah sensor tekanan seharusnya menghasilkan data pada rentang

$$80 \leq P \leq 120 \text{ kPa.}$$

Nilai  $-1$  digunakan sebagai sentinel, sedangkan nilai lain di luar rentang valid harus ditolak. Buat program yang menghitung rata-rata, minimum, maksimum, dan jumlah data yang menyimpang lebih dari 10 kPa dari tekanan referensi 100 kPa.

Program harus tetap benar jika beberapa input pertama tidak valid. Jelaskan secara eksplisit bagaimana nilai valid pertama digunakan untuk menginisialisasi minimum dan maksimum.

### 5.30.8 Algoritma iteratif dengan dua kriteria penghentian

Rancang algoritma yang memperbarui nilai menurut hubungan

$$x_{k+1} = \frac{1}{2} \left( x_k + \frac{A}{x_k} \right).$$

Pengulangan harus berhenti jika

$$|x_{k+1} - x_k| < \varepsilon$$

atau jika jumlah iterasi telah mencapai `max_iteration`. Implementasikan dalam C dan tunjukkan melalui dua eksperimen mengapa keberadaan batas iterasi maksimum merupakan perlindungan yang penting.

### 5.30.9 Estimasi integral dengan penjumlahan Riemann

Gunakan pola akumulator untuk mendekati integral

$$\int_0^1 x^2 dx$$

menggunakan jumlah Riemann titik tengah. Bagi interval menjadi  $N$  bagian sama lebar dan hitung

$$\Delta x = \frac{1}{N}.$$

Untuk setiap subinterval, evaluasi fungsi pada titik tengah lalu akumulasi kontribusinya. Uji untuk  $N = 10, 100, 1000$ , dan  $10000$ , kemudian bandingkan hasil numerik dengan nilai eksak  $1/3$  dan analisis perubahan galat.

### 5.30.10 Proyek mini: analisis data getaran mesin

Kembangkan studi kasus getaran mesin pada catatan ini menjadi program yang menerima data percepatan sampai sentinel `9999`. Program harus menolak data di luar rentang  $-50$  sampai  $50 \text{ m/s}^2$  dan menghitung rata-rata, RMS, minimum, maksimum, jumlah data dengan  $|a| > 8 \text{ m/s}^2$ , serta persentasenya.

Tambahkan satu fitur baru: hitung **crest factor**

$$C = \frac{\max |a_i|}{a_{\text{RMS}}}.$$

Program harus menghitung `max_abs` selama proses berlangsung tanpa menyimpan seluruh data. Lakukan tracing manual untuk satu himpunan data uji, bandingkan dengan keluaran program, lalu jelaskan mengapa rata-rata, RMS, dan crest factor memberikan informasi yang berbeda mengenai sinyal getaran.

## 5.31 Rangkuman

- ▶ Pola algoritma iteratif membantu kita mengenali struktur penyelesaian masalah yang berulang pada banyak kasus komputasi. Dengan mengenali pola, kita dapat merancang algoritma sebelum memikirkan detail sintaks.
- ▶ Setiap iterasi mempunyai keadaan awal, aturan pembaruan, dan kondisi penghentian. Ketiga bagian tersebut harus dirancang secara konsisten agar algoritma benar dan berhenti.
- ▶ *Counter* digunakan untuk menghitung banyak kejadian. Nilai awalnya biasanya nol karena belum ada kejadian yang tercatat sebelum proses dimulai.
- ▶ *Akumulator/accumulator* digunakan untuk mengumpulkan nilai secara bertahap. Untuk penjumlahan, nilai awal yang alami adalah nol.
- ▶ Rata-rata dapat diperoleh dengan menggabungkan akumulator dan informasi banyak data. Pembagian hanya boleh dilakukan jika banyak data lebih besar dari nol.
- ▶ Minimum dan maksimum sebaiknya diinisialisasi menggunakan data valid pertama. Pendekatan tersebut lebih aman daripada menggunakan konstanta seperti nol.

- ▶ Sentinel digunakan untuk menandai akhir masukan ketika jumlah data tidak diketahui sebelumnya. Nilai sentinel harus diperiksa sebelum data diproses agar tidak ikut masuk ke perhitungan.
- ▶ Validasi masukan memastikan nilai memenuhi syarat matematis atau fisik sebelum digunakan. Validasi dapat digabungkan dengan sentinel, counter, akumulator, serta pola lainnya.
- ▶ Pengulangan bersarang digunakan ketika persoalan melibatkan lebih dari satu indeks atau kombinasi parameter. Jika dua loop masing-masing berukuran  $N$ , jumlah operasi dapat bertumbuh sebagai  $O(N^2)$ .
- ▶ Tabel komputasi dapat dibuat dengan pengulangan untuk mempelajari perilaku model fisika terhadap waktu atau parameter. Pencacah integer sering berguna untuk menghasilkan langkah real yang konsisten.
- ▶ Algoritma iteratif numerik menggunakan nilai iterasi sebelumnya untuk menghasilkan nilai baru. Bentuk umumnya dapat ditulis sebagai

$$x_{k+1} = f(x_k).$$

- ▶ Kriteria penghentian dapat didasarkan pada toleransi, ukuran galat, ukuran perubahan, atau jumlah iterasi maksimum. Dalam praktik, toleransi sering digabungkan dengan batas iterasi maksimum agar program tetap aman.
- ▶ Ukuran langkah memengaruhi ketelitian dan biaya komputasi pada algoritma berbasis langkah tetap. Langkah kecil dapat meningkatkan resolusi tetapi membutuhkan lebih banyak iterasi.
- ▶ Tracing merupakan alat penting untuk memeriksa perubahan keadaan algoritma. Pemeriksaan harus dilakukan menggunakan kasus biasa, kasus batas, data tidak valid, dan interpretasi fisika.
- ▶ Banyak statistik sederhana dapat dihitung dalam satu kali penelusuran data. Pendekatan tersebut sering memiliki kompleksitas waktu  $O(N)$  dan kebutuhan memori tambahan  $O(1)$ .
- ▶ Pada kuliah berikutnya kita akan mempelajari larik, fungsi, dan modularisasi program. Konsep tersebut memungkinkan kita menyimpan banyak data secara terstruktur dan memecah program besar menjadi bagian-bagian yang lebih mudah diuji serta digunakan kembali.